

Lecture 13: The Network Layer — Forwarding and Routing

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

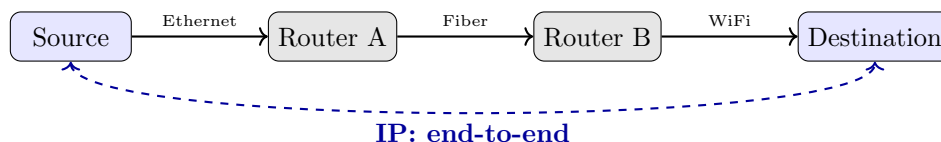
By the end of this lecture, you should be able to:

1. Explain the three responsibilities of the network layer: addressing, forwarding, and routing
2. Describe what a router is, and explain the range from home routers to Tier-1 backbone hardware
3. Explain the end-to-end argument and why IP was deliberately designed as best-effort
4. Place X.25 and ATM as historical alternatives to IP, and explain why IP won
5. Distinguish the data plane from the control plane, and explain why the separation matters
6. Explain the difference between forwarding (local, per-packet) and routing (network-wide, algorithmic)
7. Describe router architecture at a functional level: input ports, switching fabric, output ports, and queuing
8. Apply longest-prefix match to find the correct forwarding table entry for a destination address
9. Read and interpret a real routing table from the command line
10. Explain what SDN is, where it is deployed today, and what problem it solves

1 From Links to the Network

The course has so far covered the physical and data link layers of the protocol stack. The physical layer (Lectures 2–4) describes how bits are encoded onto a medium — electrical signals on copper, radio waves through air, optical pulses in fiber. The data link layer (Lectures 5–9) describes how frames are transferred reliably between **directly connected** nodes: error detection, MAC addressing, Ethernet, WiFi. Both of these layers operate one hop at a time. Ethernet delivers a frame from your laptop to the campus router; WiFi delivers a frame from your phone to the access point. Neither protocol has any concept of “across the internet.”

The **network layer** solves the harder problem: getting a packet from a source host to a destination host that may be many hops away, connected through many different networks that use many different link technologies.

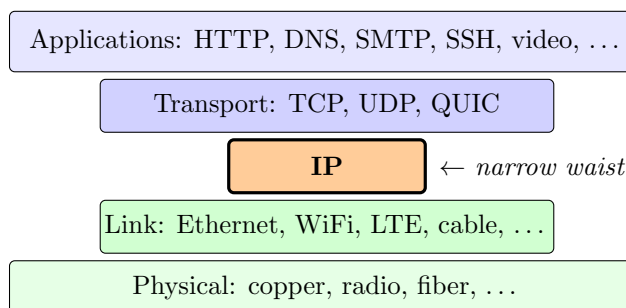


Each arrow above represents a different link technology. The network layer provides a **common abstraction** that makes end-to-end delivery work regardless of what the underlying links are. Three key properties define that abstraction:

- **Best-effort delivery.** IP does not guarantee delivery, ordering, or timing. A packet may be dropped, duplicated, or arrive out of order. Reliability is the job of the transport layer (TCP), not the network layer. This is a deliberate design choice, not an oversight — the justification is covered in Section 3.
- **Connectionless.** Each packet is forwarded independently. There is no setup phase, no circuit, and (in the traditional model) no per-flow state in the network.
- **Universal.** IP runs on every device — routers, hosts, phones, IoT sensors — and it runs over every link technology. This universality is what makes IP the “narrow waist” of the internet.

The Narrow Waist

The internet’s architecture is often described as an **hourglass** or **narrow waist**.¹ IP is the single protocol everything runs on top of, and that runs on top of everything. The figure below shows the five-layer model with IP at the narrow middle:



IP’s simplicity is a feature, not a bug. By being best-effort and connectionless, IP is trivial to implement on any new link technology and lets each transport protocol make its own choices about reliability, ordering, and timing.

2 Three Responsibilities of the Network Layer

The network layer has three distinct jobs. Each occupies its own part of the lecture unit.

1. Addressing.² Every host on the internet must have a globally unique identifier so we can specify where a packet should go. IPv4 uses 32-bit addresses written in dotted decimal

¹Wikipedia: Hourglass model — the architectural role of IP as the universal substrate; everything above runs on IP, and IP runs on everything below.

²Wikipedia: Internet Protocol — IP design philosophy, packet format, best-effort and connectionless operation.

(e.g., 10.1.2.5); IPv6 uses 128-bit addresses written in colon-hexadecimal. CIDR (Classless Inter-Domain Routing) notation combines an address with a prefix length to denote a range: 10.1.2.0/24 means the 256 addresses from 10.1.2.0 to 10.1.2.255. Addressing is covered in detail in Lecture 14.

2. Forwarding. At each router, given an incoming packet, decide which output port to send it out on. Forwarding is a **data-plane** function: it happens per-packet, in nanoseconds, using the forwarding table. A router that processes a 100 Gb/s input stream must make a forwarding decision every ~ 120 ns for 1500-byte packets.

3. Routing. Across the network, compute what the forwarding tables throughout the network should contain so that packets reach their destinations efficiently. Routing is a **control-plane** function: it runs over seconds to minutes, using algorithms such as Dijkstra’s shortest-path algorithm (Lecture 15), the Bellman-Ford algorithm (Lecture 16), and the Border Gateway Protocol (Lecture 17).

Two other network-layer mechanisms are essential in practice but are not covered today. **DHCP** (Dynamic Host Configuration Protocol) automatically assigns an IP address to a host when it joins a network; it is covered in Lecture 18. **DNS** (Domain Name System) translates human-readable names like `google.com` into IP addresses like `142.250.80.46`; it is an application-layer protocol covered later in the course.

What Is a Router?

A **router** is a device that forwards packets between networks at Layer 3 (the IP layer).³ This distinguishes it from a **switch**, which forwards frames within a single network at Layer 2. Routers span an enormous range of scale, cost, and capability:

Type	Example hardware	Forwarding rate
Home / SOHO	Netgear, TP-Link, Apple	~ 1 Gb/s
Campus edge	Cisco ASR 1000	10–100 Gb/s
ISP edge	Cisco ASR 9000, Juniper MX	100 Gb/s–1 Tb/s
Tier-1 backbone	Cisco CRS-X, Juniper PTX	1–10+ Tb/s

Where routers live matters for understanding the network as a whole. Your **home router** bundles many functions in one box: it forwards packets between your LAN and the ISP, runs a DHCP server to assign addresses to your devices, performs network address translation (NAT), and acts as a basic firewall. A **campus edge router** (at BU, for example) connects the campus network to upstream ISPs and to commodity internet exchange points. **ISP core routers** aggregate traffic from thousands of customers. **Tier-1 backbone routers**⁴ sit at major internet exchange points (Equinix, DE-CIX) where the networks of large ISPs interconnect.

³Wikipedia: Router (computing) — router hardware, forwarding tables, routing protocols, and how routers differ from switches.

⁴Wikipedia: Tier 1 network — the handful of ISPs (AT&T, Lumen, NTT, Deutsche Telekom, etc.) that form the internet’s core fabric; they peer directly with each other at major exchange points and have no upstream provider.

3 Why Best-Effort? The End-to-End Argument

Best-effort delivery is not a limitation or oversight. It is a deliberate architectural decision with a well-articulated theoretical justification: the **end-to-end argument**, first stated by Jerome Saltzer, David Reed, and David Clark in a landmark 1984 paper that remains one of the most cited works in networking.⁵

Key Result

End-to-End Argument (informal). If a function requires application-specific knowledge to implement correctly, it **must** be implemented at the endpoints. Doing it inside the network is redundant — the application still has to check anyway. Therefore, implementing it inside the network only adds complexity and cost without eliminating the endpoint check.

Applied to reliability: even if every router along the path verified delivery of each packet, your application would still need to confirm that: (1) the data arrived at the correct process, not merely the correct machine; (2) the data was interpreted and acted on correctly; and (3) the session is still valid. The network cannot know what “received correctly” means for an arbitrary application. Therefore, implement reliability once, in TCP, at the endpoints.

The internet achieves reliability through **layered, end-to-end checking**. The link layer (below IP) performs CRC error detection hop-by-hop and 802.11 WiFi retransmits frames lost on the wireless link. IP itself does nothing about reliability: if a packet’s header checksum fails, the router drops the packet silently. TCP (above IP) provides ordered, reliable, end-to-end byte-stream delivery — sequencing, acknowledgment, retransmission, flow control, and congestion control. This is not a workaround; it is the design. Each layer handles what it is best placed to handle.

4 The Alternatives: X.25, ATM, and the Design Lesson

Best-effort was not the only option. Two major alternatives tried to build reliability and quality-of-service guarantees *into* the network fabric, and both failed to displace IP.

X.25 (1976–1990s).⁶ X.25 was a connection-oriented, reliable, sequenced packet delivery standard at the network layer. Every switch along the path maintained per-connection state and performed error recovery. It was the ITU standard that serious networks used while ARPANET and IP were seen as experimental. It was dominant in Europe and enterprise through the 1980s, the backbone of many national PTT (postal, telephone, and telegraph authority) networks. It is largely obsolete since the mid-1990s.

ATM (Asynchronous Transfer Mode, 1980s–1990s).⁷ ATM used fixed 53-byte cells (a compromise between voice and data that pleased nobody) and hardware-switched them at line rate, with quality-of-service guarantees built into the switching fabric. Around 1992, ATM was widely described as “the future of all networking” — backed by every major telecom company, the ATM Forum, IEEE, and ITU. The 53-byte cell size imposed a 10% header overhead (5 bytes of header

⁵Wikipedia: End-to-end principle — statement of the argument, applications to reliability, encryption, and other network functions; the Saltzer/Reed/Clark paper.

⁶Wikipedia: X.25 — ITU connection-oriented packet-switched network standard; per-hop error recovery; dominant in European and enterprise networks through the 1980s.

⁷Wikipedia: Asynchronous Transfer Mode — fixed 53-byte cells, hardware-switched, QoS guarantees in the network; designed for integrated voice and data; lost to IP over Fast/Gigabit Ethernet.

on 48 bytes of payload). ATM lost decisively to IP over Fast Ethernet and Gigabit Ethernet and was mostly gone by 2005.

A note on OSI.⁸ The OSI model does not mandate best-effort. OSI Layer 3 (Network) explicitly supports both connection-oriented operation (as in X.25) and connectionless operation (as in IP). The TCP/IP community made the explicit choice to go connectionless. OSI's Transport Layer was designed to compensate for whatever quality the underlying network provided: Transport Class 0 was for reliable networks like X.25 (minimal transport services needed); Class 4 was for unreliable networks like IP (essentially what we call TCP today).

The outcome of the IP vs. X.25/ATM competition illustrates a lasting principle:

Key Result

“Good Enough” Beats “Perfect.” A simple, deployable design that can be iterated on quickly will often outcompete a theoretically superior design that is complex, slow to standardize, or expensive to implement. IP won not because it was technically optimal but because it was simple enough to run on commodity hardware, implement quickly on any new link technology, and deploy without per-connection state.

The **telecom mindset** (AT&T, ITU, European PTTs) held that the network should guarantee everything — reliability, ordering, QoS, billing — so that applications could be simple. Get the specification right before deploying. The **internet mindset** (ARPANET, IETF) held that the network should be as simple as possible, complexity pushed to endpoints, running on cheap commodity hardware, iterating fast.

IP/Ethernet won because commodity hardware costs fell with Moore's Law, the simple design could be implemented quickly on any new link technology (Token Ring today, WiFi tomorrow, LTE the day after), open standards enabled competition, no per-connection state meant it scaled to billions of hosts, and once IP had network effects, no migration path existed for the alternatives. The complex systems lost because per-connection state in every switch is expensive and fragile at scale, ITU and ISO standards bodies moved slowly while the IETF deployed working systems, and QoS guarantees designed for voice traffic did not match what data applications needed.

Richard Gabriel's essay **“Worse is Better”** (1991) identified this pattern in programming language design and named the underlying principle: an approach that prioritizes simplicity of implementation over correctness and completeness — the “worse” design — gets deployed, iterated on, accumulates users and momentum, and ultimately wins against the “right thing” design that optimizes for completeness but is hard to build.

The pattern recurs. **SDN** (Software-Defined Networking) is the cloud industry rediscovering the value of centralized network-level coordination — a local version of what X.25 and ATM tried to do across the whole internet. It works precisely because it is scoped to a single organization that controls all its infrastructure. **5G network slicing** is mobile telecom trying again to build QoS guarantees into the network, promising dedicated virtual “slices” with different latency, bandwidth, and reliability properties for different applications. The irony: 5G network slicing is largely implemented using SDN and NFV (Network Function Virtualization) running on top of commodity IP infrastructure.⁹ Even when telecom gets its way, it builds on the “good enough” IP stack.

⁸Wikipedia: OSI model — the seven-layer reference model; Layer 3 supports both connection-oriented (X.25) and connectionless (IP) operation; Transport Layer classes 0–4 are designed to compensate for different network qualities.

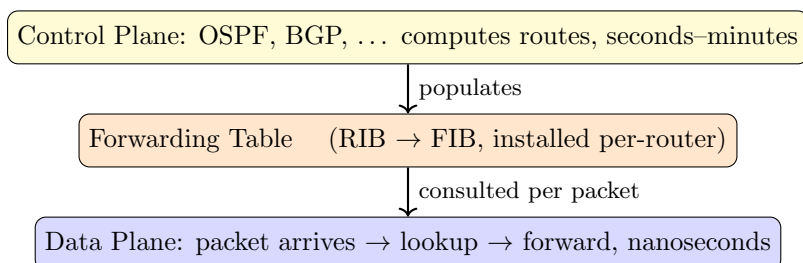
⁹Wikipedia: 5G network slicing — virtual network partitioning in 5G; QoS-in-the-network, implemented on SD-

5 Data Plane vs. Control Plane

Every router performs two distinct functions that operate on fundamentally different timescales and require fundamentally different hardware.^{10,11}

The **data plane** (fast path) operates at line rate: nanoseconds to microseconds per packet. Its job is simple: receive a packet, look at the destination IP address, consult the forwarding table, send the packet out the correct interface. This is implemented in specialized hardware (ASICs, NPUs) in any high-speed router. It is dumb but fast — it just executes the table.

The **control plane** (slow path) operates over seconds to minutes. Its job is to build and maintain the forwarding tables that the data plane uses. It runs routing protocols such as OSPF, BGP, and RIP, and it can run on a general-purpose CPU because correctness matters more than speed. It is smart but slow — it computes the strategy.



The separation matters for three reasons. First, **failure isolation**: if the routing protocol crashes or is slow to converge, packets already in flight continue to be forwarded using the last-known-good tables. The data plane does not depend on the control plane being responsive at any given instant. Second, **different optimization targets**: data-plane hardware is optimized for throughput and latency; control-plane software is optimized for correctness, flexibility, and expressiveness. Putting both on the same processing unit would force a compromise that serves neither goal well. Third, **enables SDN**: if the control plane is cleanly separated from the data plane, there is no reason it must run inside the router at all. It can run anywhere — including on a server with a global view of the network. This is the fundamental insight behind Software-Defined Networking (Section 9).

RIB vs. FIB. The control plane builds and maintains the **Routing Information Base (RIB)**: the full routing table as computed by routing protocols, which may contain many entries with redundancy, preferences, and policy annotations. From the RIB, the router installs a subset into the **Forwarding Information Base (FIB)**: a hardware-optimized structure (typically stored in TCAM, see Section 8) that the data plane actually consults per packet. The FIB is a fast, pruned view of the RIB.

N/NFV over IP.

¹⁰Wikipedia: Forwarding plane — the data plane (fast path), implemented in ASICs; per-packet processing.

¹¹Wikipedia: Control plane — the control plane (slow path), routing protocols, RIB vs. FIB distinction.

Demo

Reading a routing table with `ip route show`.

The routing table is the interface between control plane and data plane — the control plane writes it, and the data plane reads it for every packet. On a Linux host:

```
$ ip route show
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.42 metric 100
10.0.0.0/8 via 10.1.0.1 dev eth1 proto static
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.42
```

Reading each line:

- `default via 192.168.1.1 dev eth0` — for any destination not matched by a more specific entry, send to gateway `192.168.1.1` via interface `eth0`. `default` is shorthand for `0.0.0.0/0`, the prefix of length zero that matches every address.
- `10.0.0.0/8 via 10.1.0.1 dev eth1 proto static` — for destinations in the `10.x.x.x` range, forward via `10.1.0.1` on `eth1`. The `proto static` tag means this entry was added by an administrator, not by a routing daemon.
- `192.168.1.0/24 dev eth0 ... scope link` — the local subnet; these hosts are directly reachable on `eth0` without a gateway.

The control plane (DHCP client, static configuration, or routing daemon) installed these entries. The data plane uses them for every forwarded packet.

A portable alternative that works on both macOS and Linux: `netstat -rn`.

6 Forwarding vs. Routing

The terms *forwarding* and *routing* are often used interchangeably in casual speech, but they have precise and distinct meanings in networking.

Forwarding asks: given this specific packet, which output port should I send it on? This is a local question, answered by a single router, using its forwarding table. It is a data-plane operation that must be completed in nanoseconds. The analogy is following GPS turn-by-turn directions: at each intersection, you act on the next instruction without knowing the full route.

Routing asks: what should the forwarding tables across the entire network say, so that packets reach their destinations efficiently? This is a network-wide computation, performed by routing algorithms and protocols running in the control plane over seconds to minutes. The analogy is computing the route before you leave: Dijkstra's algorithm, Bellman-Ford, or BGP path selection finds the path; the resulting forwarding tables are the turn-by-turn instructions.

Example**Step-by-step forwarding in a router's data plane.**

For each arriving packet, the data plane executes the following sequence:

1. Receive bits on the input interface; reconstruct the IP packet from the incoming frame (strip the link-layer header and trailer).
2. Check the IP header checksum. If the checksum fails, drop the packet silently. Decrement the **TTL** field by 1. Recompute and update the header checksum (TTL is in the checksum-covered header).
3. If the new TTL equals 0: drop the packet and send an **ICMP Time Exceeded** message back to the source. This prevents routing loops from circulating packets indefinitely.
4. Look up the destination IP address in the forwarding table using **longest-prefix match** (Section 8). This yields an output interface and possibly a next-hop address.
5. Send the packet out the matching output interface. If the link is Ethernet, **ARP** is used to resolve the next-hop IP address to a MAC address if the mapping is not already cached.

TTL as loop prevention.¹² The Time to Live field is an 8-bit counter initialized by the source host. Typical initial values are 64 (Linux default), 128 (Windows default), and 255 (Cisco routers). Because each router decrements the TTL by exactly 1, the observed TTL in a received packet lets you estimate the number of hops it has traveled: $\text{hops} = \text{initial TTL} - \text{observed TTL}$. When you see a reply from **google.com** with $\text{TTL} = 54$, and you know Google's Linux servers use an initial TTL of 64, you can infer the path was approximately 10 hops.

¹²Wikipedia: Time to live — TTL field in IPv4 headers; decremented at each hop; when it reaches zero the packet is dropped and an ICMP Time Exceeded message is sent.

Demo

Discovering hops with traceroute.^a

traceroute exploits the TTL mechanism to map the path between two hosts. It sends a series of probes with TTL = 1, 2, 3, ... Each router that receives a probe with TTL = 1 decrements it to 0, drops the packet, and sends back an ICMP Time Exceeded message, thereby revealing its own address. By incrementing the TTL one step at a time, **traceroute** collects the addresses of successive routers along the path.

```
$ traceroute google.com
traceroute to google.com (142.250.80.46), 64 hops max
 1  192.168.1.1      1.2 ms    <- home router / default gateway
 2  10.24.0.1       8.4 ms    <- ISP edge router
 3  72.14.222.1     14.3 ms   <- Google peering point
 4  142.251.48.85   15.1 ms
 5  142.250.80.46   15.6 ms   <- destination
```

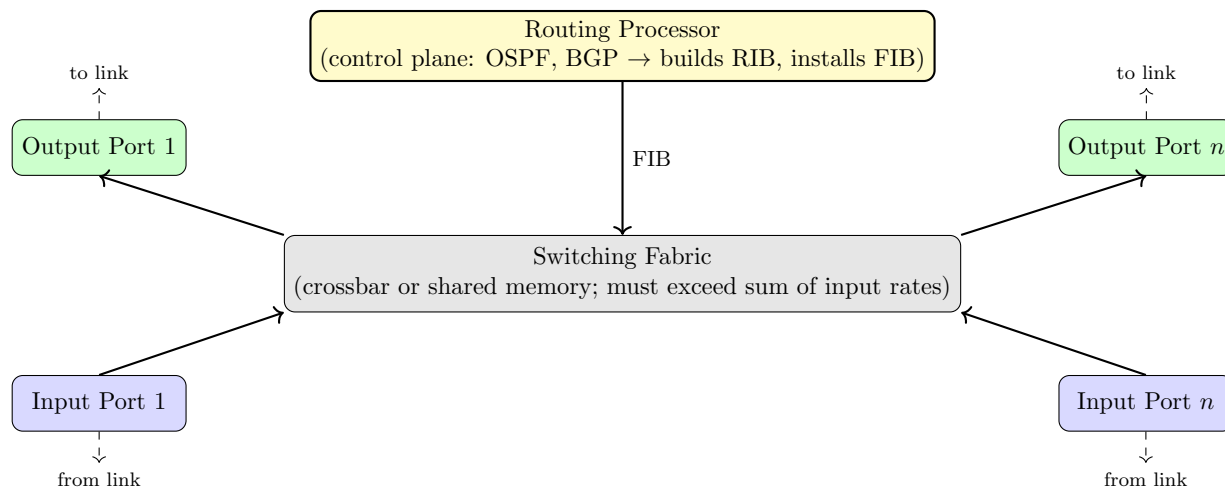
Critically: the routers are not doing anything special for **traceroute**. They are simply executing their normal data-plane forwarding logic (steps 2–3 in the example above). **traceroute** is an emergent behavior from the TTL mechanism, not a special protocol.

A line showing * * * means that the router at that hop silently drops TTL-expired packets (a common firewall policy) rather than sending an ICMP reply. It does **not** mean the packet was dropped at that hop for all traffic, and it does not indicate that the end-to-end path is broken.

^aWikipedia: Traceroute — TTL exploitation to discover path hops; ICMP Time Exceeded replies; * * * firewall behavior.

7 Router Architecture

Understanding router architecture explains why forwarding must be in hardware, why queuing and packet loss are unavoidable, and why the control plane is physically separate. A Cisco CRS-X or Juniper PTX forwards tens of terabits per second. The engineering choices that make this possible are visible in the architecture.



Input ports (one per physical interface) perform three functions: the physical layer receives bits and recovers the clock; the link layer strips the incoming frame header and checks the FCS; and a copy of the FIB is consulted to determine the output port before the packet is forwarded to the switching fabric. A small queue buffers packets briefly while the fabric is busy.

The switching fabric interconnects all input and output ports. It must be able to transfer data faster than the aggregate rate of all input ports combined, or it becomes a bottleneck. In high-end routers, this is a **crossbar switch**: a grid of $n \times n$ crosspoints, each able to connect any input to any output simultaneously. Lower-cost routers may use a shared memory bus, which is simpler but ultimately limited by memory bandwidth.

Output ports receive packets from the switching fabric, queue them to absorb bursts, add the outgoing link-layer header (e.g., an Ethernet header for the next hop), and transmit bits onto the physical medium.

The **routing processor** is the control plane in hardware form. It runs OSPF, BGP, and other routing protocols, maintains the RIB, and periodically installs updated FIB entries into the input-port line cards.

Queuing and Packet Loss

Queues form at output ports whenever packets arrive from the switching fabric faster than the output link can drain them. When a queue overflows, packets are dropped. This is the fundamental mechanism of congestion in the internet: routers signal overload by discarding packets, which TCP senders detect and respond to by reducing their transmission rate.

The default drop policy is **tail drop**: when the queue is full, the most recently arriving packet is discarded. **RED (Random Early Detection)**¹³ is an active queue management policy that begins dropping packets *probabilistically* before the queue is completely full, giving TCP senders an early congestion signal and avoiding the “global synchronization” problem where all senders reduce their window at the same time.

Head-of-line (HOL) blocking¹⁴ arises at input ports with FIFO queuing: if a packet at the front of an input queue is waiting for a busy output port, it blocks all the packets behind it in that queue, even those destined for output ports that are currently free. The fix is **Virtual Output Queues (VOQ)**: maintain a separate queue at each input port for each output port. This eliminates HOL blocking at the cost of more complex queue management.

Why Forwarding Must Be Hardware

A 100 Gb/s Ethernet link delivers a minimum-sized frame (64 bytes) every 5 ns and a maximum-sized frame (1500 bytes) every ~120 ns. Sequential DRAM lookup through 900,000 routing table entries is impossible at these rates. The solution is **TCAM (Ternary Content-Addressable Memory)**: a specialized memory that stores each forwarding entry as a (value, mask) pair, where each bit can be 0, 1, or **X** (don’t care). A don’t-care bit naturally represents the host portion of a prefix. TCAM searches **all entries in parallel** in a single clock cycle. It is expensive in power and die area, and the capacity is limited — the full BGP routing table (approximately 900,000 prefixes) barely fits in commodity TCAM chips. This is an active engineering constraint.

¹³Wikipedia: Random early detection — active queue management that drops packets probabilistically before the queue fills, providing early congestion signals to TCP.

¹⁴Wikipedia: Head-of-line blocking — packet at the head of an input queue blocks packets behind it that could use a different, free output port; fixed by virtual output queues.

8 Longest-Prefix Match

IP addresses are organized hierarchically: prefixes aggregate large blocks of addresses. A routing table entry for 10.0.0.0/8 covers 16 million addresses; an entry for 10.1.2.0/24 covers only 256. A destination address can therefore match multiple entries of different prefix lengths simultaneously.¹⁵

Key Result

Longest-Prefix Match Rule. When a destination address matches more than one forwarding table entry, always use the entry with the **longest** (most specific) matching prefix. Longer prefix = more specific routing instruction. A more specific entry was installed precisely to handle that narrower address range and should take precedence over the coarser default.

Example

Applying longest-prefix match.

Consider the following forwarding table:

Prefix	Length	Next Hop	Interface
0.0.0.0/0	0	203.0.113.1	eth0 (default)
10.0.0.0/8	8	10.255.0.1	eth1
10.1.0.0/16	16	10.1.255.1	eth2
10.1.2.0/24	24	10.1.2.254	eth3
192.168.0.0/16	16	192.168.0.1	eth4

Destination 10.1.2.5: Matches 0.0.0.0/0 (length 0), 10.0.0.0/8 (length 8), 10.1.0.0/16 (length 16), and 10.1.2.0/24 (length 24). Longest match: 10.1.2.0/24 → forward out **eth3**.

Destination 10.5.6.7: Matches 0.0.0.0/0 (length 0) and 10.0.0.0/8 (length 8). Does not match 10.1.0.0/16 (the first two octets match but not the third). Longest match: 10.0.0.0/8 → forward out **eth1**.

Destination 172.16.0.1: Matches only 0.0.0.0/0 (length 0). Default route → forward out **eth0**.

¹⁵Wikipedia: Longest prefix match — the LPM rule, its role in IP forwarding, and CIDR lookup implementation.

Demo**Longest-prefix match in Python with the `ipaddress` module.**

Python's standard library `ipaddress` module makes prefix matching straightforward. The containment operator `dest in prefix` returns `True` if `dest` falls within the prefix range. The longest match is the entry with the maximum `prefixlen`.

```
import ipaddress

# Build a simple forwarding table
table = [
    (ipaddress.ip_network("0.0.0.0/0"),      "eth0"),
    (ipaddress.ip_network("10.0.0.0/8"),      "eth1"),
    (ipaddress.ip_network("10.1.0.0/16"),     "eth2"),
    (ipaddress.ip_network("10.1.2.0/24"),     "eth3"),
    (ipaddress.ip_network("192.168.0.0/16"),  "eth4"),
]

def longest_prefix_match(dest_str, table):
    dest = ipaddress.ip_address(dest_str)
    matching = [p for p, _ in table if dest in p]
    best = max(matching, key=lambda p: p.prefixlen)
    return best, dict(table)[best]

for dest in ["10.1.2.5", "10.5.6.7", "172.16.0.1", "192.168.100.1"]:
    prefix, iface = longest_prefix_match(dest, table)
    print(f"{dest:20s} -> {str(prefix):20s} via {iface}")
```

If no entry matches (no default route) and the destination is outside all prefixes, the `max()` call fails — the router would drop the packet and generate an ICMP Destination Unreachable message.

In hardware, longest-prefix match is performed by **TCAM**¹⁶ (described in Section 5): each entry is stored as a (value, mask) pair, the ternary X bits represent the host portion of the prefix, and a single clock cycle suffices to identify all matching entries. The priority encoder then selects the longest match.

9 Software-Defined Networking

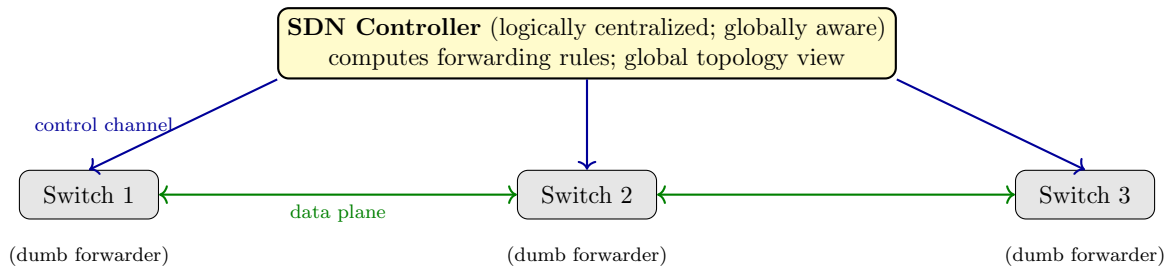
The traditional internet architecture embeds the control plane in every router. OSPF routers exchange link-state advertisements and each independently runs Dijkstra to compute the same global shortest-path tree. BGP routers exchange path advertisements and each independently applies local routing policy. This distributed model has worked for decades, but it has a fundamental limitation: each router has only a local view of the network. It cannot easily enforce network-wide traffic engineering policies, react to topology changes in a globally coordinated way, or optimize aggregate link utilization.

Software-Defined Networking (SDN)¹⁷ is built on the data/control plane separation intro-

¹⁶Wikipedia: Ternary content-addressable memory — ternary bits (0, 1, X); parallel search across all entries in one clock cycle; expensive in power and area; used for FIB lookup in high-speed routers.

¹⁷Wikipedia: Software-defined networking — SDN overview, centralized control plane, OpenFlow protocol, adop-

duced in Section 5. If the control plane is cleanly separated from the data plane, it does not need to run inside the router. It can run on a server — or a cluster of servers — that has a **global view** of the entire network. Data-plane devices become simple forwarding engines that execute instructions from the controller.



OpenFlow¹⁸ is the original SDN protocol: the controller pushes flow table rules to switches (“if destination IP matches 10.1.2.0/24, send out port 3”). Switches just match packets against their rules and act. There is no routing protocol in the switch.

Where SDN actually runs. SDN is deployed at scale inside **cloud data centers**. **Google B4**¹⁹ (published 2013) replaced dedicated hardware routers with commodity switches running a centralized traffic engineering controller for Google’s WAN between data centers. The result was near-100% link utilization versus the 30–40% achievable with OSPF, saving enormous infrastructure cost. **AWS VPC** uses SDN for all virtual networking inside AWS: the controller knows about every virtual machine and pushes forwarding rules to hypervisors. There is no traditional routing protocol between your instances. Meta and Microsoft Azure have similar fabric architectures.

SDN does not run the open internet. BGP and OSPF still run the internet’s core. There is no internet-wide SDN controller, and there is unlikely to be one: you cannot give any single entity control over the routing of the entire internet, which is owned by thousands of independent organizations. SDN was predicted circa 2012 to replace all networking within a few years. It transformed cloud networks; it has not displaced the open internet’s routing architecture.

SDN is the cloud industry rediscovering centralized coordination — a local version of what X.25 and ATM tried to do across the whole internet. The difference is scope. When one organization controls all the infrastructure (a data center, a campus, an enterprise WAN), central control is practical, beneficial, and used in production. When the infrastructure is owned by thousands of independent parties, it is not.

tion in data centers.

¹⁸Wikipedia: OpenFlow — the original SDN protocol; controller pushes flow table rules to switches; match/action model.

¹⁹Wikipedia: Software-defined wide area network — SD-WAN; Google B4 uses SDN to achieve near-100% WAN link utilization vs. 30–40% with traditional OSPF.

Demo

Cloud SDN in practice: VPC route tables.

If you have access to AWS CLI or Google Cloud CLI, routing inside a VPC (Virtual Private Cloud) is pure control-plane configuration — there is no routing protocol. The controller installed every route when you created subnets, VPC peerings, or VPN connections.

```
# AWS: view route tables for a VPC (pure SDN control plane)
aws ec2 describe-route-tables --region us-east-1

# Google Cloud: list routes
gcloud compute routes list
```

The routes you see were programmatically installed by the cloud SDN controller. No OSPF, no BGP, no link-state advertisements — just the controller pushing entries in response to your configuration actions.

Contrast with the traditional internet: when you run `ip route show` on a Linux router participating in OSPF, the entries tagged `proto ospf` were installed by a routing daemon running Dijkstra's algorithm. The entries tagged `proto dhcp` were installed by the DHCP client when it received a lease. In both cases the control plane wrote the table; the data plane reads it.

Further Reading

The following Wikipedia articles are the primary student-facing references for this lecture. They are suitable for reviewing concepts, exploring topics in greater depth, and following citations to original papers.

- Wikipedia: Network layer — OSI and TCP/IP network layer overview; responsibilities of Layer 3.
- Wikipedia: Internet Protocol — IP design philosophy, packet format, best-effort and connectionless operation.
- Wikipedia: Hourglass model — the narrow waist; IP as the universal substrate of the internet.
- Wikipedia: End-to-end principle — Saltzer, Reed, Clark (1984); justification for best-effort design; applications to reliability, encryption, and other network functions.
- Wikipedia: Router (computing) — router hardware, forwarding table, routing protocols, router vs. switch distinction.
- Wikipedia: Tier 1 network — backbone ISPs; peering; internet exchange points (Equinix, DE-CIX).
- Wikipedia: X.25 — connection-oriented network layer; per-hop error recovery; historical alternative to IP.
- Wikipedia: Asynchronous Transfer Mode — ATM; fixed 53-byte cells; QoS in the network; why it lost to IP.

- Wikipedia: OSI model — seven-layer reference model; Layer 3 connection-oriented vs. connectionless; Transport Layer classes.
- Wikipedia: Forwarding plane — data plane (fast path); ASICs; per-packet processing.
- Wikipedia: Control plane — routing protocols; RIB vs. FIB distinction; slow path.
- Wikipedia: Longest prefix match — LPM algorithm; CIDR forwarding; priority among matching entries.
- Wikipedia: Ternary content-addressable memory — TCAM; ternary bits; parallel lookup in one clock cycle; power and area tradeoffs.
- Wikipedia: Traceroute — TTL exploitation; ICMP Time Exceeded; * * * firewall behavior.
- Wikipedia: Time to live — TTL field in IPv4; loop prevention; estimating hop count.
- Wikipedia: Software-defined networking — SDN overview; centralized control; OpenFlow; adoption in data center and WAN networks.
- Wikipedia: OpenFlow — flow tables; controller-switch protocol; match/action model.
- Wikipedia: Software-defined wide area network — SD-WAN; Google B4; WAN traffic engineering.
- Wikipedia: Network function virtualization — NFV; virtualizing network functions on commodity hardware; relationship to SDN.
- Wikipedia: 5G network slicing — virtual network partitioning with QoS guarantees; implemented via SDN/NFV over IP.
- Wikipedia: Head-of-line blocking — HOL blocking in switch input queues; virtual output queues as the fix.
- Wikipedia: Random early detection — RED active queue management; early congestion signals for TCP.

Prepared for EC 441, Boston University, Spring 2026.